JSSMAHAVIDYAPEETHA



Mini Project/Internship Assessment

on

"CODET: An AI-Powered Code Quality Intelligence Tool"

Subject Name: Mini Project/Internship Assessment
Subject Code: BCS-752

COURSE: B.Tech.
SEMESTER: VII

by

Abhay Shukla (Roll No. _____)

Department of Computer Science and Engineering –AIML
JSS ACADEMY OF TECHNICAL EDUCATION C-20/1,
SECTOR-62, NOIDA

VISIONANDMISSION

VISION OF THE INSTITUTE

JSS Academy of **T**echnical **E**ducation Noida aims to become an Institution of excellence in imparting quality **O**utcome **B**ased **E**ducation that empowers the young generation with **K**nowledge, **S**kills, **R**esearch, **A**ptitude and **E**thical values to solve **Contemporary Challenging Problems.**

MISSION OF THE INSTITUTE

Develop a platform for achieving globally acceptable level of intellectual acumen and technological competence.

Create an inspiring ambience that raises the motivation level for conducting quality research.

Provide an environment for acquiring ethical values and positive attitude.

VISION OF THE DEPARTMENT

“To spark the imagination of the Computer Science Engineers with values, skills and creativity to solve the real-world problems.”

MISSION OF THE DEPARTMENT

To inculcate creative thinking and problem-solving skills through effective teaching, learning and research.
To empower professionals with core competency in the field of Computer Science and Engineering.
To foster independent and lifelong learning with ethical and social responsibilities.

PROGRAM OUTCOMES (POs)

Engineering Graduates will be able to:

PO1: Engineering knowledge: Apply the knowledge of mathematics, science, engineering fundamentals, and an engineering specialization to the solution of complex engineering problems.

PO2: Problem analysis: Identify, formulate, review research literature, and analyze complex engineering problems reaching substantiated conclusions using first principles of mathematics, natural sciences, and engineering sciences.

PO3: Design/development of solutions: Design solutions for complex engineering problems and design system components or processes that meet the specified needs with appropriate consideration for the public health and safety, and the cultural, societal, and environmental considerations.

PO4: Conduct investigations of complex problems: Use research-based knowledge and research methods including design of experiments, analysis and interpretation of data, and synthesis of the information to provide valid conclusions.

PO5: Modern tool usage: Create, select, and apply appropriate techniques, resources, and modern engineering and IT tools including prediction and modeling to complex engineering activities with an understanding of the limitations.

PO6: The engineer and society: Apply reasoning informed by the contextual knowledge to assess societal, health, safety, legal and cultural issues and the consequent responsibilities relevant to the professional engineering practice.

PO7: Environment and sustainability: Understand the impact of the professional engineering solutions in societal and environmental contexts, and demonstrate the knowledge of, and need for sustainable development.

PO8: Ethics: Apply ethical principles and commit to professional ethics and responsibilities and norms of the engineering practice.

PO9: Individual and teamwork: Function effectively as an individual, and as a member or leader in diverse teams, and in multidisciplinary settings.

PO10: Communication: Communicate effectively on complex engineering activities with the engineering community and with society at large, such as, being able to comprehend and write effective reports and design documentation, make effective presentations, and give and receive clear instructions.

PO11: Project management and finance: Demonstrate knowledge and understanding of the engineering and management principles and apply these to one’s own work, as a member and leader in a team, to manage projects and in multidisciplinary environments.

PO12: Life-long learning: Recognize the need for, and have the preparation and ability to engage in independent and life-long learning in the broadest context of technological change.

PROGRAM EDUCATIONAL OUTCOMES (PEOs)

PEO1: To apply computational skills necessary to analyze, formulate and solve engineering problems.

PEO2: To establish as entrepreneurs, and work in interdisciplinary research and development organizations as an individual or in a team.

PEO3: To inculcate ethical values and leadership qualities in students to have a successful career.

PEO4: To develop analytical thinking that helps them to comprehend and solve real-world problems and inherit the attitude of lifelong learning for pursuing higher education.

PROGRAM SPECIFIC OUTCOMES (PSOs)

PSO1: Acquiring in depth knowledge of theoretical foundations and issues in Computer Science to induce learning abilities for developing computational skills.

PSO2: Ability to analyse, design, develop, test and manage complex software system and applications using advanced tools and techniques.

COURSEOUTCOMES(COs)

C340.1	Developing a technical artifact requiring new technical skills and effectively Utilizing a new software tool to complete a task.
C340.2	Writing requirements documentation, Selecting appropriate technologies, Identifying and creating appropriate test cases for systems.
C340.3	Demonstratingunderstandingofprofessionalcustoms&practicesandworking With professional standards.
C340.4	Improving problem- solving, critical thinking skills and report writing.
C340.5	Learning professional skills like exercising leadership, behaving professionally, behaving ethically, listening effectively, participating as a member of a team, developing appropriate workplace attitudes.

CO-PO-PSOMAPPING

	PO1	**PO2**	**PO3**	**PO4**	**PO5**	**PO6**	**PO7**	**PO8**	**PO9**	**PO10**	**PO11**	**PO12**	**PSO1**	**PSO2**
C340.1	3	3	3	3	2	3	3	3	3	3	2	3	3	3
C340.2	3	3	3	3	3	3	3	3	3	2	3	3	3	3
C340.3	2	2	3	3	3	2	3	3	3	1	2	3	3	3
C340.4	2	2	2	2	2	2	2	2	2	3	2	3	2	2
C340.5	2	2	2	2	2	2	2	2	2	3	2	3	2	2
C340	2.40	2.40	2.60	2.60	2.40	2.40	2.60	2.60	2.60	2.40	2.20	3.00	2.60	2.60

DECLARATION

I hereby declare that this submission is my own work and that, to the best of my knowledge and belief, it contains no material previously published or written by another person nor material which to a substantial extent has been accepted for the award of any other degree or diploma of the university or other institute of higher learning, except where due acknowledgment has been made in the text.

Signature
: Name :
Roll No.:
Date :

CERTIFICATE

This is to certify that Mini Project/Internship Assessment Report entitled “CODET: An AI-Powered Code Quality Intelligence Tool” which

is submitted by Abhay Shukla in partial fulfillment of the requirement for the award of degree B. Tech. in Department of Computer Science and Engineering of Dr. APJ Abdul Kalam Technical University, Uttar Pradesh, Lucknow is a record of the candidate’s own work carried out by him/her under my supervision. The matter embodied in this report is original and has not been submitted for the award of any other degree.

Supervisor **Date**

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to the Department of Computer Science and Engineering (Artificial Intelligence and Machine Learning), JSS Academy of Technical Education, Noida, for giving me the opportunity to undertake this mini project as a part of the B.Tech. VII semester curriculum.

I am deeply thankful to my project supervisor for the constant guidance, constructive feedback and encouragement provided throughout the course of this work. Their suggestions on system design and on the evaluation of AI-generated results helped shape the project into its present form.

I extend my thanks to the Head of the Department and all faculty members of the department for providing the academic environment and laboratory facilities required for the successful completion of this project.

I also acknowledge the open-source community behind Python, FastAPI, LangChain, Qdrant, Redis, React and the many libraries on which this project is built. Their documentation and tooling made it possible to build a fairly complex system within the limited time frame of a semester.

Finally, I am grateful to my family and friends for their continuous support and motivation.

Abhay Shukla
B.Tech. CSE (AIML), VII Semester

ABSTRACT

Modern software projects grow quickly in size and complexity, and keeping their quality under control is a recurring challenge for developers and reviewers. Conventional static analysis tools such as linters and rule-based scanners are fast and precise for the patterns they know, but they cannot reason about intent, cross-file behaviour or project-specific conventions, and they offer little help when a developer simply wants to understand an unfamiliar codebase.

This report presents **Codet**, an AI-powered code quality intelligence tool that combines large language models (LLMs), retrieval-augmented generation (RAG) and a multi-agent architecture to analyse source code written in Python, JavaScript and TypeScript. An **Orchestrator Agent** receives a structural tree of the repository and strategically selects which files to inspect, while a **File Analysis Agent** examines each selected file and reports issues across eight categories (security, performance, duplication, complexity, testing, documentation, style and maintainability) with five severity levels, line numbers and concrete fix suggestions. The agents cooperate through a capability-scoped **shared memory** consisting of notes, todos with an explicit lifecycle, and a per-file analysis cache.

Large repositories are automatically parsed into semantic chunks using the Python AST and Tree-sitter, embedded with two sentence-transformer models (one for natural language and one for code) and stored in the Qdrant vector database, enabling hybrid semantic search during analysis and interactive question answering. Organisation-specific coding rules written in Markdown are indexed in the same way and retrieved per file. Conversation history and caches are kept in Redis.

The system is delivered through two interfaces: a rich command-line interface built with Click and Rich, and a web application consisting of a FastAPI backend with REST and WebSocket endpoints and a React dashboard that streams live agent progress, visualises the results and allows the user to chat with the codebase. The tool can run against Google Gemini in the cloud or a local model served by Ollama. The resulting reports include a prioritised issue list, category and severity breakdowns and a weighted quality score, giving developers an actionable and explainable view of their code quality.

Keywords: Code Quality, Static Analysis, Large Language Models, Multi-Agent Systems, Retrieval-Augmented Generation, Vector Database, FastAPI, React.

ABOUT THE INSTITUTE

JSS Academy of Technical Education (JSSATE), Noida was established in 1998 by the **JSS Mahavidyapeetha**, Mysuru, an educational trust that runs a large network of schools, colleges and universities across India and abroad. The institute is located at C-20/1, Sector-62, Noida, Uttar Pradesh, is approved by the All India Council for Technical Education (AICTE), New Delhi, and is affiliated to **Dr. A.P.J. Abdul Kalam Technical University (AKTU)**, Lucknow.

The institute offers undergraduate (B.Tech.) programmes in several engineering disciplines along with postgraduate programmes, and aims to impart quality outcome-based education that empowers students with knowledge, skills, research aptitude and ethical values, as reflected in its vision and mission statements presented at the beginning of this report.

The **Department of Computer Science and Engineering (Artificial Intelligence and Machine Learning)** offers a four-year B.Tech. programme that combines a strong foundation in computer science with specialised courses in machine learning, deep learning, natural language processing, data analytics and related areas. The department encourages students to apply these technologies to real-world problems through laboratory work, mini projects, internships and the final-year major project.

The mini project described in this report was carried out in the department as a part of the course **Mini Project/Internship Assessment (BCS-752)** in the VII semester. The project gave an opportunity to work with current AI technologies such as large language models, vector databases and agent frameworks, and to integrate them into a complete, usable software product.

TABLE OF CONTENTS

Title **Page No.**

Declaration.....vi
Certificate.....vii
Acknowledgement.....viii
Abstract.....ix
About the Institute.....x
List of Tables.....xiii
List of Figures.....xiv
List of Abbreviations.....xv
Chapter 1: Introduction.....1
1.1 Overview of the Project.....1
1.2 Project Background.....1
1.3 Problem Statement.....2
1.4 Objectives of the Project.....2
1.5 Scope of the Project.....3
1.6 My Role in the Project.....3
Chapter 2: Tools and Technology Used.....4
2.1 Python.....4
2.2 LangChain.....5
2.3 Large Language Models: Google Gemini and Ollama.....5
2.4 Qdrant Vector Database.....5
2.5 Python AST and Tree-sitter.....6
2.6 Redis.....6
2.7 FastAPI and WebSockets.....7
2.8 Comparative Study and Justification.....7
Chapter 3: History and Features of the Technology.....8
3.1 History of Static Code Analysis.....8
3.2 History of Language Models for Code.....8
3.3 Retrieval-Augmented Generation.....9
3.4 AI Agents and Multi-Agent Systems.....9
3.5 The Paradigm Shift.....9
Chapter 4: Work Done.....11

4.1 System Architecture....11
4.2 Use Case Diagram....12
4.3 Data Flow Diagrams....13
4.4 Module Information....15
4.5 Important Algorithms....20
4.6 Screenshots and Discussion....21
4.7 Demonstration and Testing....26
Chapter 5: Conclusions and Future Scope....27
5.1 Conclusion....27
5.2 Use of the Learned Technology in the Final Year Major Project....27
5.3 Industrial Relevance....28
5.4 Societal Relevance and Impact....28
5.5 Limitations....28
5.6 Future Scope....28
References....29

LIST OF TABLES

Table No. and Title **Page No.**

Table 2.1 Technology stack of the project....4
Table 2.2 Comparison of code analysis approaches....7
Table 4.1 Backend modules and their responsibilities....16
Table 4.2 Shared memory permissions by agent role....18
Table 4.3 REST and WebSocket endpoints of the backend....19
Table 4.4 Results of the demonstration runs on the taskboard repository....26

LIST OF FIGURES

Figure No. and Title **Page No.**

Figure 4.1 System architecture of Codet....12
Figure 4.2 Use case diagram of Codet....13
Figure 4.3 Level 0 DFD (context diagram)....13
Figure 4.4 Level 1 DFD....14
Figure 4.5 Level 2 DFD for the file analysis process....15
Figure 4.6 Landing page of the Codet web interface....21
Figure 4.7 Folder upload and live analysis progress in the web interface....22
Figure 4.8 Analysis dashboard for the taskboard repository....23
Figure 4.9 Agent events while a chat question is being answered....24
Figure 4.10 Answer to a question about SQL injection in the codebase....24
Figure 4.11 Live orchestrator events during a CLI analysis....25
Figure 4.12 Summary printed by the CLI after the analysis....25

LIST OF ABBREVIATIONS

Abbreviation	Full Form
AI	Artificial Intelligence
AICTE	All India Council for Technical Education
AKTU	Dr. A.P.J. Abdul Kalam Technical University
API	Application Programming Interface
AST	Abstract Syntax Tree
CLI	Command Line Interface
DFD	Data Flow Diagram
HTTP	HyperText Transfer Protocol
JSON	JavaScript Object Notation
LLM	Large Language Model
NLP	Natural Language Processing
Q&A	Question and Answer
RAG	Retrieval-Augmented Generation
REST	Representational State Transfer
UI	User Interface
URL	Uniform Resource Locator
WS	WebSocket

CHAPTER 1
INTRODUCTION

1.1 Overview of the Project

Software quality is one of the most important factors that decides whether a project can be maintained, extended and trusted over time. As a codebase grows, defects, security weaknesses, duplicated logic and undocumented behaviour accumulate silently. Code reviews and automated linters help, but reviewers have limited time and linters only recognise the specific patterns they have been programmed for. A large part of the knowledge needed to judge code quality - what a function is supposed to do, whether a check is missing in some other file, whether a convention of the team is being followed - remains implicit.

Codet (short for *Code Quality Intelligence Tool*) is a mini project that attempts to close this gap by using large language models as reasoning engines over source code. Instead of matching fixed patterns, Codet reads the code the way a human reviewer would, guided by the structure of the repository, by semantic search over the whole codebase, and by organisation-specific rules supplied by the user. The output is a structured, prioritised quality report together with an interactive question-answering facility that lets a developer ask natural-language questions about the code.

The tool is implemented in Python on the backend and React on the frontend. It can be used either from the terminal through the `codet` command or through a web application in which a user can paste a GitHub repository URL or upload files and folders, follow the progress of the AI agents in real time, and explore the resulting report on a dashboard.

1.2 Project Background

Static analysis has a long history, from the original `lint` program for C to modern tools such as Pylint, ESLint, Bandit and SonarQube. These tools work by parsing source code and evaluating rules over the syntax tree or a control-flow representation. Their strengths are speed, determinism and a low false-negative rate for well-understood problems. Their weaknesses are equally well known: rules must be written by hand for each language and each problem, findings often lack context, the number of false positives can be high, and they cannot answer

questions such as *"where is the user authenticated and is that check applied to every API route?"*

1.3 Problem Statement

The problem addressed by this project can be stated as follows: *design and implement a system that accepts a source code repository (local or on GitHub), automatically identifies code quality issues across multiple categories with precise locations and actionable suggestions, scales to repositories that do not fit into the context window of a language model, respects project-specific coding rules, and allows developers to interactively ask questions about the codebase.*

Several sub-problems follow from this statement:

- **Context limits** - a language model can only read a limited number of tokens at once, so the system must decide which files are worth reading and must retrieve only the relevant parts of large codebases.
- **Coordination** - findings in one file often depend on another file (for example, a function that is never tested), so information discovered by one analysis step must be available to later steps.
- **Structured output** - the free-form text produced by a model must be converted into validated, machine-readable issues that can be counted, sorted and displayed.

1.4 Objectives of the Project

The main objectives of the project are:

1. To build a multi-agent analysis engine in which an orchestrator agent plans the analysis of a repository and delegates individual files to a specialised file analysis agent.
2. To detect and classify issues into eight categories (security, performance, duplication, complexity, testing, documentation, style and maintainability) with five severity levels, line numbers, code snippets and suggested fixes.
3. To implement automatic indexing of large repositories into a vector database using language-aware chunking (Python AST and Tree-sitter for JavaScript/TypeScript) and dual embeddings for hybrid search.

4. To support custom coding rules supplied as Markdown files, retrieved semantically for each analysed file.
5. To implement a shared memory through which agents can exchange notes, todos and cached results under role-based permissions.
6. To provide a command-line interface and a web interface (FastAPI + React) with live progress streaming, report visualisation, JSON export and an interactive chat over the analysed codebase.
7. To allow the use of either a cloud model (Google Gemini) or a locally hosted model (Ollama) so that private code need not leave the developer's machine.

1.5 Scope of the Project

The current version of Codet supports Python, JavaScript and TypeScript source files. It can analyse a single file, a local directory, an uploaded set of files or folders, or a public GitHub repository that is cloned on demand. Files and directories that are normally irrelevant to quality analysis - virtual environments, `node_modules`, build artefacts, caches, version-control metadata and anything listed in the repository's `.gitignore` - are filtered out automatically.

1.6 My Role in the Project

This project was designed and developed individually, from requirements and architecture to the final implementation and testing. My main responsibilities were:

- Designing the overall architecture, including the orchestrator/file-analysis agent split, the shared memory model and the RAG pipeline.
- Implementing the backend in Python: agents and tool-calling loop, repository tree construction, file filtering, repository size checking, multi-language parsers, Qdrant indexers, Redis-based message history and the report generator.
- Developing the React frontend: repository input and upload screen, live processing status, analysis dashboard, issue cards and chat panel.

CHAPTER 2
TOOLS AND TECHNOLOGY USED

Codet is a full-stack application that integrates several independent technologies. This chapter explains each of them, the role it plays in the project, the alternatives that were considered, and the reasons for the final choice. Table 2.1 summarises the complete technology stack.

Table 2.1: Technology stack of the project

Layer	Technology	Purpose in Codet
Language (backend)	Python 3.9+	Core implementation of agents, indexers, API and CLI
LLM framework	LangChain (core, google-genai, ollama)	Model abstraction, prompts, tool binding, structured output
Cloud LLM	Google Gemini (gemini-2.5-flash-lite)	Default reasoning model for analysis and chat
Local LLM	Ollama (llama3.2 by default)	Offline / private analysis
Vector database	Qdrant	Storage and search of code and rule embeddings
Embeddings	Sentence-Transformers (MiniLM-L6-v2, Jina code v2)	Natural-language and code vectors
Parsing	Python ast, Tree-sitter	Language-aware chunking of source code
Cache / history	Redis	Conversation history and response cache with TTL
Web backend	FastAPI, Uvicorn, WebSockets	REST API and live event streaming
CLI	Click, Rich	Command parsing and rich terminal output
Frontend	React 18, React Router, Axios, Chart.js	Web dashboard and chat interface

2.1 Python

Python was chosen as the primary implementation language because almost the entire modern AI ecosystem - model SDKs, LangChain, sentence-transformers, the Qdrant and Redis clients - is available first and best supported in Python. Python also offers the built-in `ast` module, which gives direct access to the syntax tree of Python programs and is used by Codet's Python parser. Features such as `asyncio`, type hints and dataclasses are used extensively: agent calls are asynchronous so that the web server remains responsive while long analyses run, and

data structures such as `CodeIssue`, `AnalysisResult`, `Note` and `Todo` are defined as dataclasses.

2.2 LangChain

LangChain is a framework for building applications on top of language models. Codet uses `langchain-core` for message types (`SystemMessage`, `HumanMessage`, `ToolMessage`), prompt templates and output parsers, and the provider packages `langchain-google-genai` and `langchain-ollama` to talk to Gemini and Ollama through a common chat-model interface. Because both providers implement the same interface, the rest of the code is independent of which model is used; switching to a local model is a single configuration flag (`use-local`).

LangChain's **tool binding** is used to expose Pydantic models (`AnalyzeFile`, `QueryFile`, `QueryCodebase`) as callable tools to the orchestrator. The model responds with structured tool calls, which Codet executes and feeds back as tool messages in a loop until the model produces a final answer. **Structured output** with Pydantic schemas is used to force the file analysis agent to return a JSON object containing a list of issues, notes and memory items.

2.3 Large Language Models: Google Gemini and Ollama

The reasoning in Codet is performed by a large language model. The default is Google's **Gemini** family accessed through the Gemini API (the configured default is `gemini-2.5-flash-lite`, chosen for its low latency and cost, large context window and native support for tool calling). The model is used with a low temperature of 0.1 so that analyses are as repeatable as possible, and with a maximum output of 8192 tokens.

For users who cannot send their source code to a cloud service, Codet supports **Ollama**, a runtime that serves open-weight models such as Meta's Llama 3.2 on the local machine. Before starting an analysis in local mode, the CLI checks that the Ollama server is reachable on port 11434 and that the requested model has been pulled, and prints clear instructions otherwise.

2.4 Qdrant Vector Database

Qdrant is an open-source vector database written in Rust. It stores vectors together with a JSON payload and supports fast approximate nearest-neighbour search using HNSW graphs, payload filtering and **named vectors** (several vectors per point). Codet uses named vectors to

store two embeddings for every code chunk - an `nlp` vector built from a natural-language description and a `code` vector built from the source itself - both compared with cosine distance. A second collection stores the chunks of the user's custom rule files. Qdrant can run locally in Docker, in memory for testing, or as a managed cloud service; Codet supports all three through configuration.

The embeddings are produced locally with two Sentence-Transformers models: **all-MiniLM-L6-v2** (384 dimensions) for natural-language descriptions and rule chunks, and **jina-embeddings-v2-base-code** (768 dimensions), trained on source code, for the code itself. Using both allows **hybrid search**: a question such as *"where are passwords hashed?"* is matched mainly against the descriptions, while a code snippet is matched against the code vectors.

Alternatives: FAISS (a library, not a server - no persistence or filtering out of the box), Chroma (simple, but fewer production features), Pinecone (managed only, proprietary), Weaviate and Milvus (feature-rich but heavier to operate), and PostgreSQL with pgvector. Qdrant was preferred for its named-vector support, payload filtering, simple Python client and free cloud tier.

2.5 Python AST and Tree-sitter

To index a codebase meaningfully it must be split into units that make sense - functions, methods, classes and modules - rather than arbitrary blocks of lines. For Python, Codet uses the standard library `ast` module to walk the syntax tree and extract each function and class with its signature, docstring, decorators and exact line range. For JavaScript and TypeScript it uses **Tree-sitter**, an incremental parsing library that produces concrete syntax trees for many languages; Codet walks these trees to extract function declarations, arrow functions assigned to variables, classes and methods, together with their JSDoc comments.

2.6 Redis

Redis is an in-memory key-value store. Codet uses it to persist **conversation sessions** of the orchestrator agent (so that chat follow-up questions keep their context), as a **response cache** for repeated prompts, and to hold completed analysis results for the web dashboard. Message history entries expire after 24 hours and cache entries after one hour by default. If Redis is not available the agents fall back to an in-memory cache, so the tool still works in a minimal setup.

Alternatives include SQLite or PostgreSQL (persistent but slower for this pattern) and Memcached (no rich data types).

2.7 FastAPI and WebSockets

The web backend is built with **FastAPI**, a modern asynchronous Python web framework based on Starlette and Pydantic, served by **Uvicorn**. FastAPI was chosen because it is natively asynchronous (important for long-running model calls), validates requests and responses automatically using the same Pydantic models used elsewhere in the project, and generates interactive OpenAPI documentation at `/docs`. **WebSocket** endpoints are used to stream agent events - tool start, tool completion, reasoning and memory updates - to the browser while an analysis or chat request is being processed.

The frontend is a **React 18** single-page application using React Router, Axios and react-syntax-highlighter; its component model maps naturally onto reusable issue and metric cards.

2.8 Comparative Study and Justification

Table 2.2 compares Codet with traditional rule-based analysers and single-prompt LLM usage (pasting code into a chatbot). Codet does not replace deterministic linters, which remain best for well-defined rule checks; it adds semantic, cross-file reasoning, explanations and conversational exploration on an open, replaceable stack.

Table 2.2: Comparison of code analysis approaches

Aspect	Rule-based analysers (Pylint, ESLint, SonarQube)	Single-prompt LLM	Codet (agents + RAG)
Detection basis	Hand-written rules	Model reasoning	Model reasoning + retrieved context + custom rules
Cross-file reasoning	Limited	Only what is pasted	Shared memory and semantic search
Large repositories	Yes	No (context limit)	Yes (automatic indexing)
Explanations and fixes	Short messages	Yes, unstructured	Yes, structured with line numbers
Interactive Q&A	No	Yes, without codebase grounding	Yes, grounded in the codebase

CHAPTER 3
HISTORY AND FEATURES OF THE TECHNOLOGY

Codet sits at the intersection of two lines of development: automated program analysis, which is several decades old, and large language models, which have matured only in the last few years. This chapter traces the history of both, describes the key features of the technologies used, and discusses the paradigm shift brought by AI-assisted code analysis.

3.1 History of Static Code Analysis

The idea of examining a program without running it is almost as old as programming itself. Compilers have always performed some checks, but the term *lint* comes from a tool written by Stephen C. Johnson at Bell Labs in 1978, which flagged suspicious constructs in C programs that the compiler accepted. Lint established the model followed by most later tools: parse the program, apply a set of heuristic rules, and report warnings.

3.2 History of Language Models for Code

The decisive change came in 2017 with the **Transformer** architecture introduced in the paper *"Attention Is All You Need"*. Transformers process entire sequences in parallel using self-attention and scale well with data and compute. This led to pre-trained models such as BERT (2018) and the GPT series. Code-specific models followed: CodeBERT (2020), OpenAI Codex (2021), which powered the first version of GitHub Copilot, and later open models such as StarCoder and Code Llama (2023).

From 2022 onwards, instruction-tuned conversational models such as ChatGPT, Claude, Gemini and Llama demonstrated that a single general-purpose model could explain code, find bugs, write tests and review pull requests in natural language. Google's Gemini family (first released in December 2023) and Meta's openly available Llama models (Llama 3.2 in 2024) are the two families supported by Codet. Modern models also support **function calling** or **tool use**, in which the model returns a structured request to call an external function - the foundation of AI agents.

3.3 Retrieval-Augmented Generation

Language models know only what was in their training data and can read only a limited amount of text per request. **Retrieval-augmented generation**, introduced by Lewis et al. in 2020, addresses both limitations by retrieving relevant documents from an external store at query time and adding them to the prompt. RAG relies on dense vector embeddings - pioneered by models such as Sentence-BERT (2019) - and on efficient approximate nearest-neighbour search.

In Codet, RAG is used twice: to find relevant code anywhere in a large repository (via the `QueryCodebase` tool) and to find the custom coding rules that apply to the file currently being analysed. This lets the system work on repositories far larger than any context window while keeping each prompt small and focused.

3.4 AI Agents and Multi-Agent Systems

An **AI agent** is a program in which a language model repeatedly decides on an action, observes its result and decides again until a goal is reached. The ReAct pattern (reasoning and acting, 2022) and native tool calling in commercial models made this practical. Frameworks such as LangChain, LangGraph, AutoGen and CrewAI provide abstractions for agents, tools and memory.

Multi-agent systems divide work between agents with different roles and prompts. A common pattern, used in Codet, is an **orchestrator-worker** design: a planner agent sees the big picture and delegates focused sub-tasks to worker agents, each of which has a smaller, specialised context. Shared memory - inspired by the persistent notes and todo lists used by modern coding assistants - allows workers to leave observations and follow-up tasks for each other.

3.5 The Paradigm Shift

Traditional static analysis answers the question *"does this code match a known bad pattern?"*. AI-assisted analysis answers a broader question: *"what would an experienced reviewer say about this code?"*. This shift has several consequences, summarised below.

- **From rules to reasoning** - instead of writing a new rule for every problem, the model generalises from its training to recognise problems it has never been explicitly told about, such as a missing authorisation check or a misleading function name.

- **From isolated warnings to explanations** - each finding comes with a description of why it matters, its likely impact, and a concrete fix, which lowers the effort needed to act on it.
- **From single files to whole codebases** - retrieval and shared memory let the analysis connect evidence across files, for example noticing that a function defined in one module is never tested.
- **From configuration files to natural-language rules** - team conventions can be written as plain Markdown and are applied semantically rather than syntactically.
- **From reports to conversations** - a developer can ask follow-up questions and explore the codebase interactively, which is especially valuable when onboarding onto an unfamiliar project.

The shift also introduces new challenges: model output is not deterministic, models can hallucinate, context windows are limited and source code may be confidential. Codet addresses these with a low temperature and strict Pydantic schemas, mandatory file and line references for every issue, selective file reading with RAG for large repositories, and support for local models.

CHAPTER 4
WORK DONE

This chapter describes the design and implementation of Codet in detail. It begins with the overall architecture and the diagrams that model the system (use case diagram and data flow diagrams), then describes each module, the important algorithms, and finally presents screenshots of the working system with a discussion of their inputs, outputs and integration with other modules.

4.1 System Architecture

Codet follows a layered architecture, shown in Figure 4.1. The **presentation layer** consists of the command-line interface and the React web dashboard. The web dashboard talks to a **FastAPI backend** over HTTP and WebSockets, while the CLI calls the engine directly. Both use the **Orchestrator Engine**, which constructs the repository tree, initialises the agents, runs the analysis flow and compiles the final report.

The **agent layer** contains the Orchestrator Agent and the File Analysis Agent, both built on a common `BaseAgent` class that wraps the language model (Gemini or Ollama), prompt templates, caching and the tool-calling loop. The agents communicate through the **Shared Memory**. The **data and retrieval layer** contains the repository tree constructor and file filter, the codebase and rules indexers backed by Qdrant, and Redis for message history.

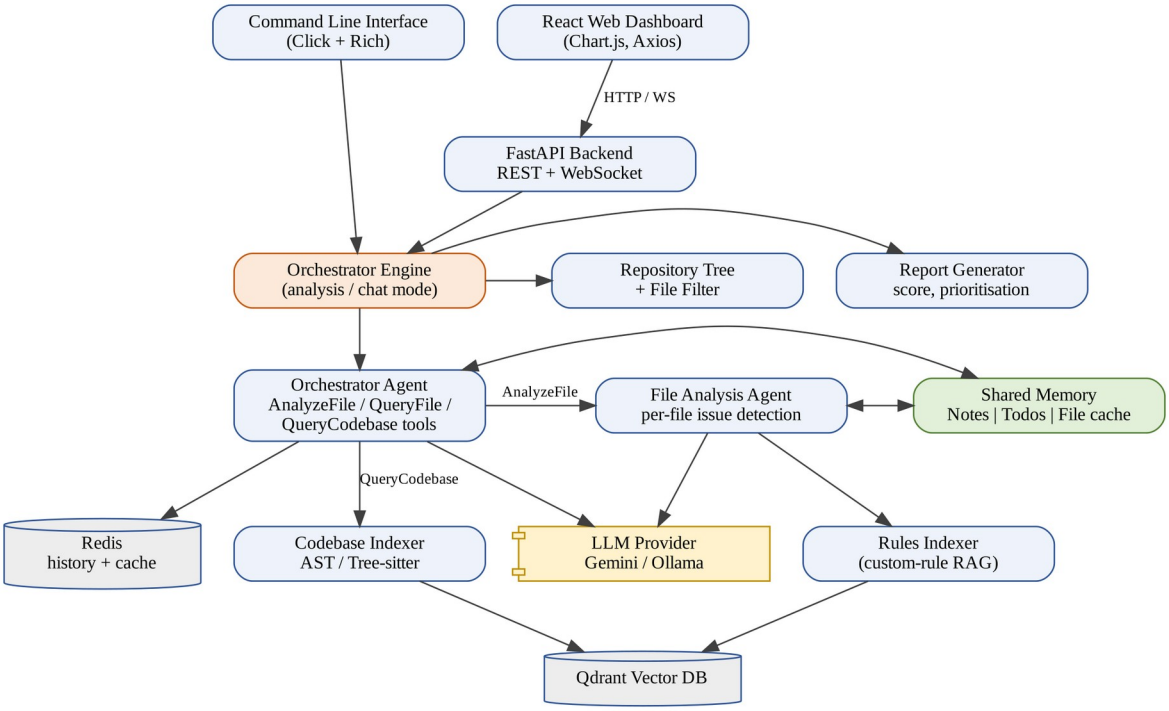


Figure 4.1: System architecture of Codet

4.2 Use Case Diagram

Figure 4.2 shows the use case diagram. The primary actor is the **developer**, who can analyse a local file or directory, analyse a GitHub repository, upload files or folders through the web interface, supply custom rule files, view the quality report and metrics, ask questions about the codebase, export the report as JSON, and follow the live progress of the agents. The **LLM service** is a secondary actor used by the analysis and chat use cases. Indexing of the codebase for RAG *extends* the analysis use case: it happens automatically for large repositories or when the user passes the `--index` flag.

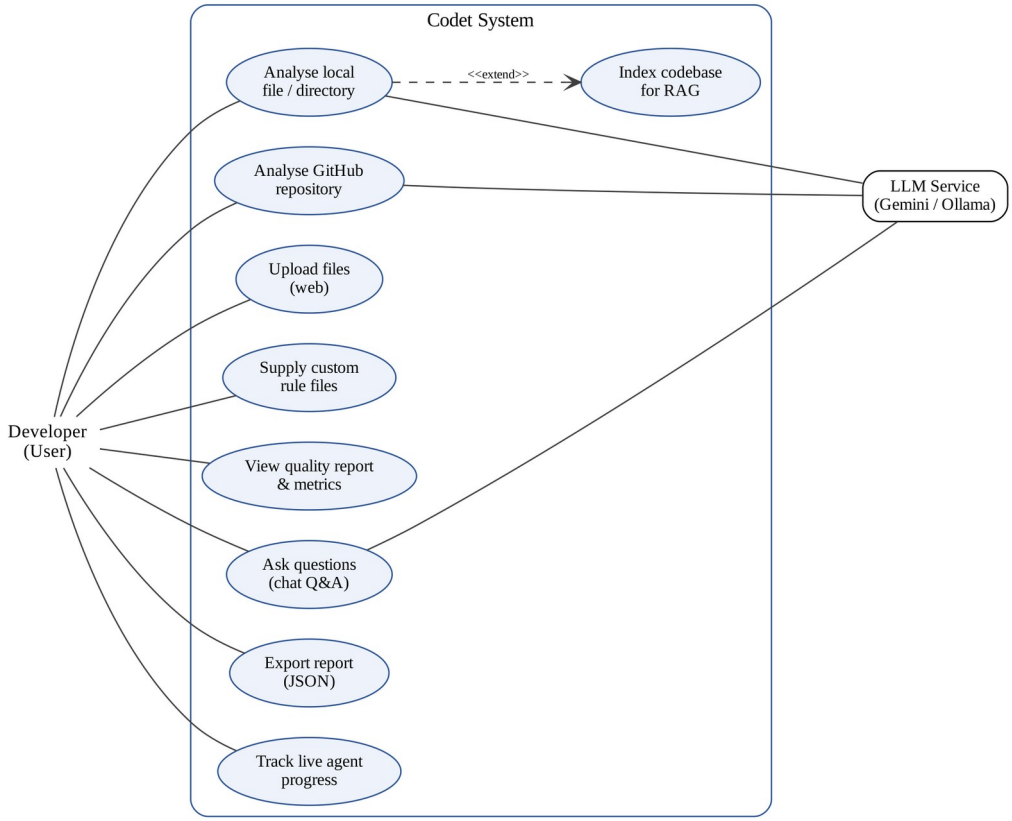


Figure 4.2: Use case diagram of Codet

4.3 Data Flow Diagrams

4.3.1 Level 0 DFD (Context Diagram)

The context diagram in Figure 4.3 treats the whole system as a single process. The developer supplies a source path or repository URL, optional rule files and questions; GitHub supplies repository files; and the LLM provider receives prompts with code context and returns structured issues, notes and answers. The system returns the quality report, metrics and answers to the developer.

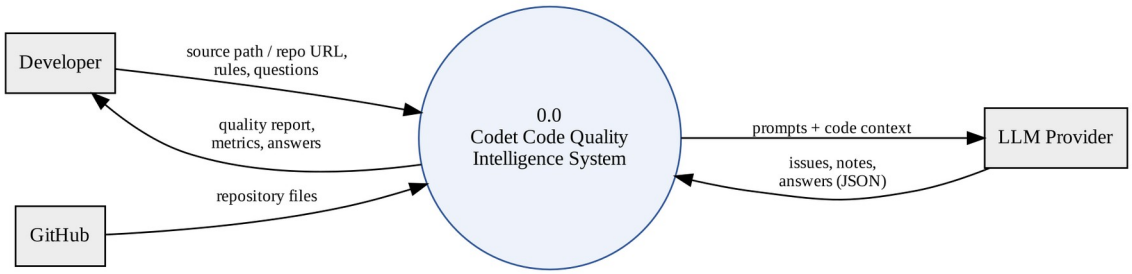


Figure 4.3: Level 0 DFD (context diagram)

4.3.2 Level 1 DFD

Figure 4.4 decomposes the system into seven processes: (1.0) acquiring the source by cloning, uploading or copying it to a temporary directory; (2.0) building the repository tree and checking the repository size; (3.0) indexing the codebase into Qdrant when required; (4.0) orchestrating the analysis; (5.0) analysing individual files with the LLM; (6.0) generating the report; and (7.0) answering follow-up questions. The data stores are the Qdrant vector collections (D1), the shared memory (D2) and the Redis message history (D3).

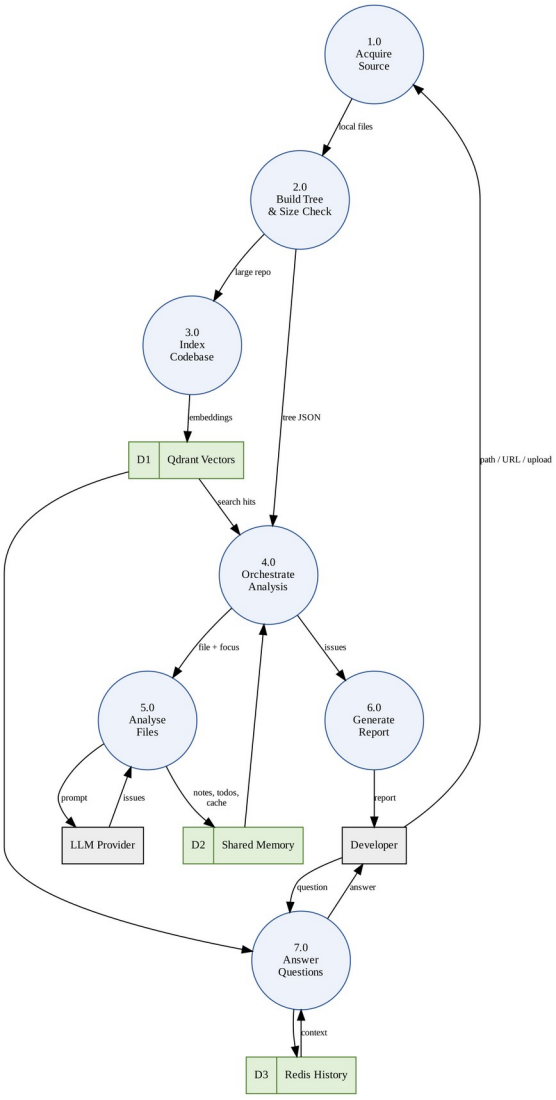


Figure 4.4: Level 1 DFD

4.3.3 Level 2 DFD (File Analysis Process)

Figure 4.5 expands process 5.0, the analysis of a single file. The file is read and trimmed to the configured limits (5.1); function and class names are extracted (5.2) and used to retrieve the most relevant custom rules from the rules collection (5.3); a prompt is assembled from the

numbered code, the retrieved rules and the current shared-memory notes and todos (5.4); the LLM is invoked with a structured-output schema (5.5); the response is validated and converted into `CodeIssue` objects (5.6); and finally new notes, todos and the file's cached result are written to shared memory (5.7).

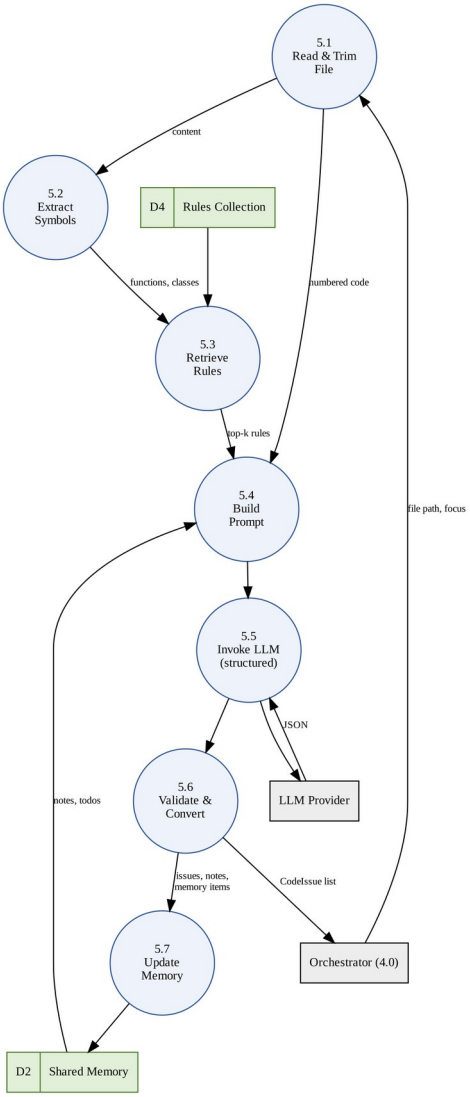


Figure 4.5: Level 2 DFD for the file analysis process

4.4 Module Information

The backend source code is organised into the packages listed in Table 4.1. Each module is described in the subsections that follow.

Table 4.1: Backend modules and their responsibilities

Package / file	Main classes	Responsibility
src/cli.py	analyze, serve commands	Command-line interface
src/core/orchestrator_engine.py	OrchestratorEngine	Runs the analysis / chat flow and compiles results
src/core/shared_memory.py	SharedMemory, MemoryView, Note, Todo	Cross-agent memory with role-based access
src/core/repository_tree.py	RepositoryTreeConstructor	Builds a JSON tree of the repository
src/core/config.py	Settings, Config, AgentConfig	Configuration from environment / .env
src/core/redis_client.py, message_history.py	RedisClient, MessageHistoryManager	Sessions, history and cache
src/agents/base_agent.py	BaseAgent	LLM wrapper, caching, tool-calling loop
src/agents/orchestrator_agent.py	OrchestratorAgent	Plans analysis, calls tools
src/agents/ file_analysis_agent.py	FileAnalysisAgent	Analyses a single file
src/agents/schemas.py, tools.py	CodeIssue, AnalysisResult, tool models	Data models and tool definitions
src/indexer/	MultiLanguageCodebaseParser, CodebaseIndexer, RulesIndexer	Parsing, embedding and retrieval
src/utils/	FileFilter, RepoSizeChecker, QdrantBase, CLIProcessingStatus	Shared utilities
src/reports/report_generator.py	ReportGenerator	Summary, quality score, prioritisation
src/api/main.py, models.py	FastAPI app	REST and WebSocket endpoints

4.4.1 Repository Tree Constructor and File Filter

The `RepositoryTreeConstructor` walks the repository and produces a nested JSON structure describing directories and files, including file sizes and languages. This tree is the orchestrator's map of the project. The `FileFilter` class centralises the rules for what to skip: a default list of patterns (compiled Python files, `__pycache__`, `.git`, `node_modules`, `.env`, logs, build and distribution folders, virtual environments and coverage output), hidden files, and the rules of the repository's own `.gitignore` parsed with `gitignore-parser`. The `RepoSizeChecker` recommends indexing when a repository has at least 100 code files or 10 MB of code; smaller repositories are analysed directly.

4.4.2 Multi-Language Parser and Codebase Indexer

The `MultiLanguageCodebaseParser` dispatches each file to a language-specific parser. The `PythonASTParser` creates a module chunk from the module docstring and a chunk for every function, method and class, recording the signature (including argument types, defaults and return annotations), docstring, decorators, start and end lines and enclosing class. The `JavaScriptTreeSitterParser` does the same for JavaScript and TypeScript, including arrow functions assigned to variables and JSDoc comments. Each chunk is represented by the `CodeChunk` Pydantic model.

The `CodebaseIndexer` embeds each chunk twice - its description with MiniLM and its code with the Jina code model - and stores it in Qdrant with named vectors `nlp` and `code`. Hybrid search merges the results of both vectors, ranking chunks found by both searches first.

4.4.3 Rules Indexer

The `RulesIndexer` lets organisations supply their own coding guidelines as Markdown files (passed with `--rules`). It splits the Markdown into chunks by headings, extracts metadata such as category, sub-category, language, priority, file patterns and keywords, embeds each chunk and stores it in a separate Qdrant collection. When a file is analysed, a query is built from the file path, the names of its functions and classes and whether it is a test file; twice the required number of candidates is retrieved and then re-ranked using keyword overlap and rule priority, as described in Section 4.5.2.

4.4.4 Shared Memory

The `SharedMemory` class is the single source of truth for information exchanged between agents. It has three layers:

- **Notes** - short, durable observations about the codebase (for example, *"authenticate() in src/auth.py is the single entry point for login"*), with optional tags and file path.
- **Todos** - action items with an explicit lifecycle: *pending*, *in_progress* and *done*, an optional owner and an optional target file (for example, *"check whether authenticate() has tests"*).
- **File cache** - the analysis result of each file, which prevents re-analysis and lets agents look up what has already been found.

Agents never modify the store directly. Each agent receives a `MemoryView`, a capability-scoped facade whose permissions depend on its role, as shown in Table 4.2. All mutations are thread-safe (guarded by a re-entrant lock), duplicate notes and todos are ignored, and every change emits a `memory_update` event that the CLI and the web interface display live.

Table 4.2: Shared memory permissions by agent role

Operation	Orchestrator	File analysis agent
Read notes, todos, file cache	Yes	Yes
Add notes and todos	Yes	Yes
Update todo status	Any todo	Own todos or todos targeting its file
Write file cache	Any file	Only the file it is analysing
Remove notes, prune todos, reset	Yes	No

4.4.5 Base Agent and Tool-Calling Loop

`BaseAgent` wraps either a `ChatGoogleGenerativeAI` or a `ChatOllama` model, prompt templates, caching and an optional Redis connection. Its `run_tool_loop` method implements the agent cycle *LLM -> tools -> LLM -> ... -> final response*: tools are bound to the model, each tool call is executed through a handler supplied by the engine, the result is appended as a tool message, and the loop ends when the model replies without tool calls. The `ReportGenerator` then computes the summary, the quality score (Section 4.5.1) and the issue priority order.

4.4.6 Orchestrator Agent

The orchestrator receives the repository tree and has access to three tools defined as Pydantic models: `AnalyzeFile` (run the file analysis agent on one file), `QueryFile` (ask a question about one file) and `QueryCodebase` (semantic search over the indexed codebase, only when indexed). Its system prompt instructs it to prioritise files referenced by pending todos, then entry points (such as `main.py`, `app.py` or `index.js`), core logic and configuration files, large or complex files, and finally tests and documentation. In **chat mode** the prompt instead asks it to analyse only the files relevant to the user's question and to answer with specific code details; previous analysis results and the Redis conversation history are available as context.

4.4.7 File Analysis Agent

The `FileAnalysisAgent` analyses one file at a time. Content beyond 1 MB is truncated and at most the first 1000 lines are analysed, with a note telling the model that the file was truncated. The code is presented with line numbers so that the model can report exact locations. Its system prompt lists the eight valid categories and asks for a clear description, line number, concrete suggestion and impact assessment for each issue. The response must conform to the `AnalysisResponseSchema`, which contains the list of issues plus `notes` and `memory_items` (new todos) for the shared memory. During conversion, unrecognised categories default to *maintainability* and unrecognised severities to *medium*, and issues that cannot be tied to a file are discarded.

4.4.8 API Module

The FastAPI application exposes the endpoints listed in Table 4.3. Uploaded files and cloned repositories are stored in temporary directories that are cleaned up afterwards. CORS is enabled for the frontend. Each analysis result is serialised and cached in Redis for one hour under an analysis identifier, which the dashboard uses to fetch the report and to open a chat session.

Table 4.3: REST and WebSocket endpoints of the backend

Method	Endpoint	Description
GET	/	Service information
GET	/api/health	Health check
POST	/api/analyze	Analyse a path on the server
POST	/api/upload	Upload files / folders for analysis
POST	/api/analyze-github	Clone and analyse a GitHub repository
POST	/api/ask/{analysis_id}	Ask a question about an analysed codebase
GET	/api/report/{analysis_id}	Fetch the report of an analysis
WS	/api/ws/analyze	Run an analysis with live event streaming
WS	/api/ws/chat/{analysis_id}	Chat with live event streaming

4.4.9 Command-Line Interface

The CLI is installed as the `codet` command. The `analyze` command accepts a file or directory path and options for output format, local models, indexing, Qdrant settings and custom rule files; the `serve` command starts the API server. While an analysis runs, a live Rich panel

shows the most recent agent events - files being analysed, codebase queries, notes added and todos created or completed.

4.4.10 Frontend Module

The React application is built from components such as `GitHubAnalyzer` (input and upload), `ProcessingStatus` (live agent events), `AnalysisDashboard` (metrics, issues and chat) and `IssueCard`. The `services/api.js` module wraps the REST calls, `utils/reportFormatter.js` converts the backend report into the shape required by the dashboard, and `utils/githubIssueHelper.js` builds pre-filled *New issue* links so that a finding can be turned into a GitHub issue with one click.

4.5 Important Algorithms

4.5.1 Quality Score Computation

Each issue is assigned a weight according to its severity: critical = 10, high = 5, medium = 2 and low/info = 1. If W is the sum of the weights of all n issues, the quality score is computed as:

```
score = max(0, 100 - (W / (10 * n)) * 100)       if n > 0
score = 100                                       if n = 0
```

The denominator $10n$ is the worst possible total (every issue critical), so the score reflects the average severity of the issues found: a codebase with only low-severity findings scores close to 90, while one dominated by critical issues approaches 0. Issues are then sorted by severity, then by category priority (security, performance, complexity, duplication, testing, maintainability, documentation, style), then by file path and line number.

4.5.2 Rule Retrieval and Re-ranking

For each analysed file, the rules indexer retrieves 2k candidate rule chunks (k = 5 by default) by cosine similarity. Each candidate's score is then adjusted: it is increased by 0.1 for every keyword of the rule that matches a function or class name in the file, and by a priority boost of +0.2 for critical rules, +0.1 for high, 0 for medium and -0.05 for low priority rules. The top k rules after re-ranking are inserted into the file analysis prompt.

4.6 Screenshots and Discussion

The screenshots in this section were captured from real runs of Codet on a small test repository, *taskboard*, described in Section 4.7. For each screen the feature of the technology used, its application in the project, its integration with other modules, and its inputs and outputs are discussed.

4.6.1 Web Interface: Landing Page

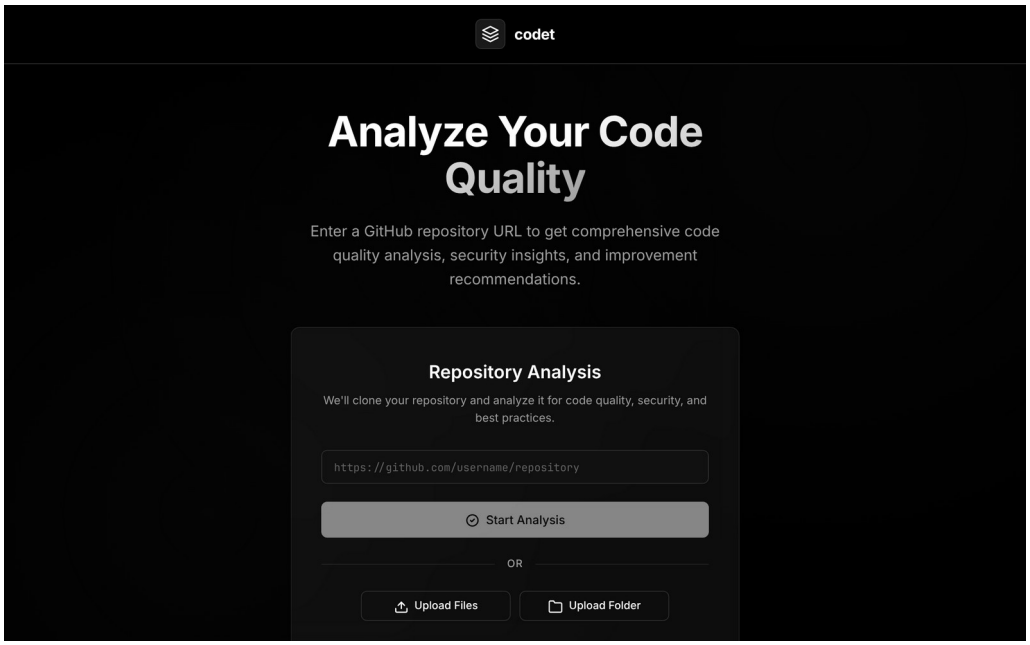


Figure 4.6: Landing page of the Codet web interface

Figure 4.6 shows the landing page rendered by the `GitHubAnalyzer` component. **Feature:** React components with state hooks render the page and manage the form state; the GitHub URL is validated on the client before any request is sent. **Application:** this is the entry point of the web workflow, where the user chooses what to analyse. **Integration:** on submission the component opens a WebSocket to `/api/ws/analyze` on the FastAPI backend, which invokes the Orchestrator Engine. **Input:** a GitHub repository URL or files/folders selected with the upload buttons. **Output:** a new analysis session.

4.6.2 Folder Upload and Live Analysis Progress

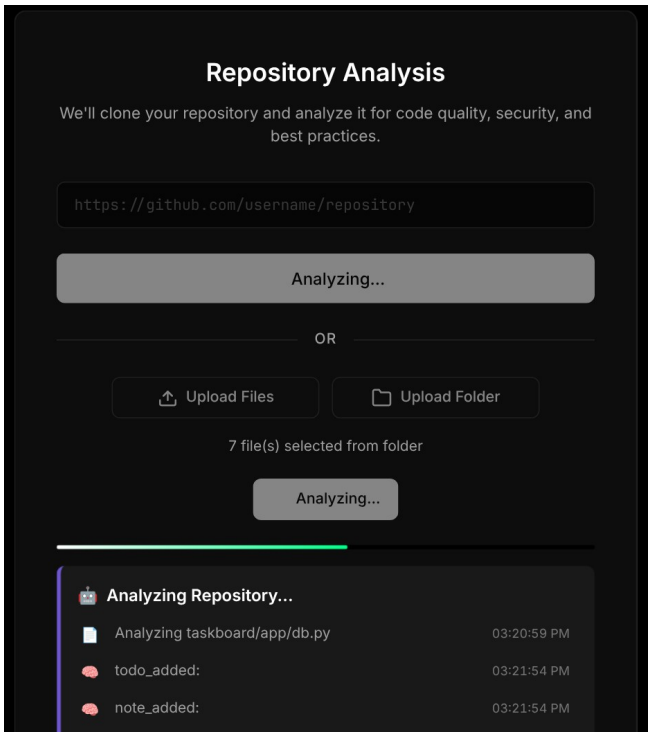


Figure 4.7: Folder upload and live analysis progress in the web interface

Figure 4.7 was captured while the *taskboard* folder was being analysed. Only supported source files are accepted (7 files were selected from the folder). **Feature:** the files are sent to `/api/upload`, and progress events are streamed back over a WebSocket. **Application:** the progress bar and event list show which file the orchestrator is analysing and when notes and todos are added to shared memory. **Integration:** the `ProcessingStatus` component renders events of type `tool_start`, `tool_complete` and `memory_update` emitted by the agents. **Input:** the uploaded files. **Output:** live progress, followed by a redirect to the dashboard.

4.6.3 Analysis Dashboard

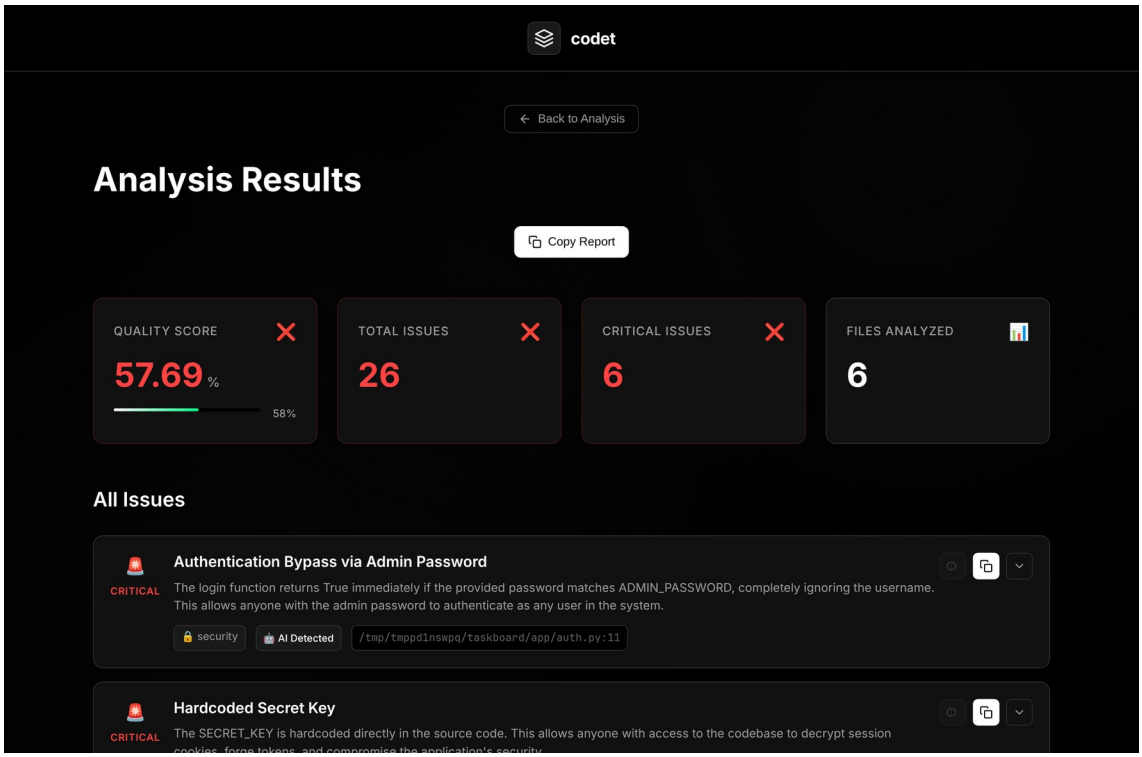


Figure 4.8: Analysis dashboard for the taskboard repository

Figure 4.8 shows the dashboard after the web analysis finished: a quality score of 57.69%, 26 issues, 6 of them critical, across 6 analysed files. The issue cards list the most severe findings first - an authentication bypass through the admin password, a hard-coded secret key and admin password, SQL injection in `find_user` and `search_tasks`, and remote code execution through `pickle.loads` - each with a description, category and exact file and line. **Feature:** React metric cards and issue cards. **Integration:** the dashboard fetches the report cached in Redis from `/api/report/{analysis_id}`. **Input:** the analysis report. **Output:** visual metrics and an interactive issue list from which GitHub issues can be created.

4.6.4 Interactive Chat with the Codebase

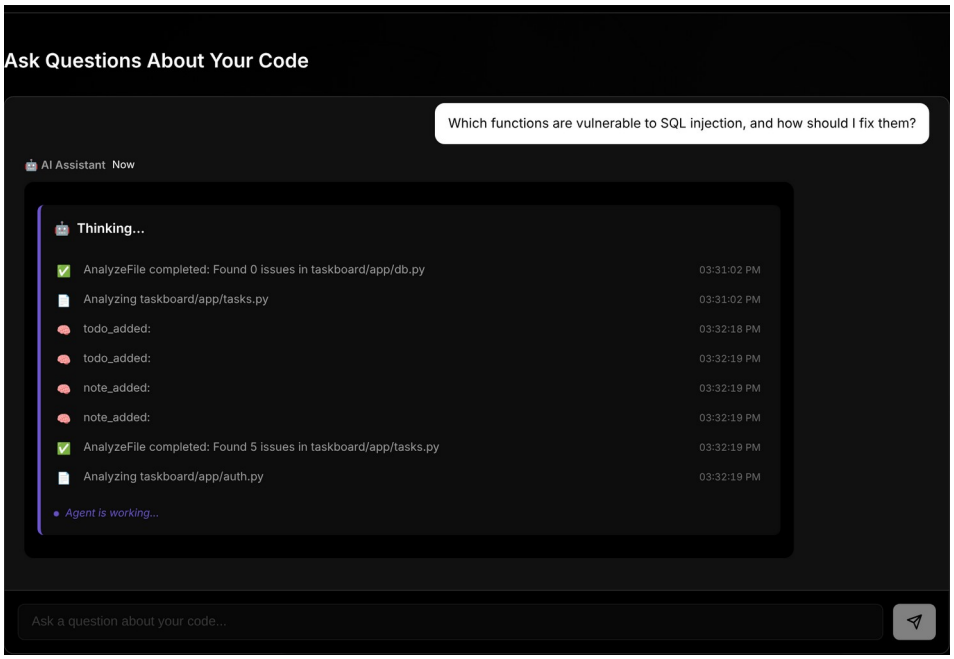


Figure 4.9: Agent events while a chat question is being answered

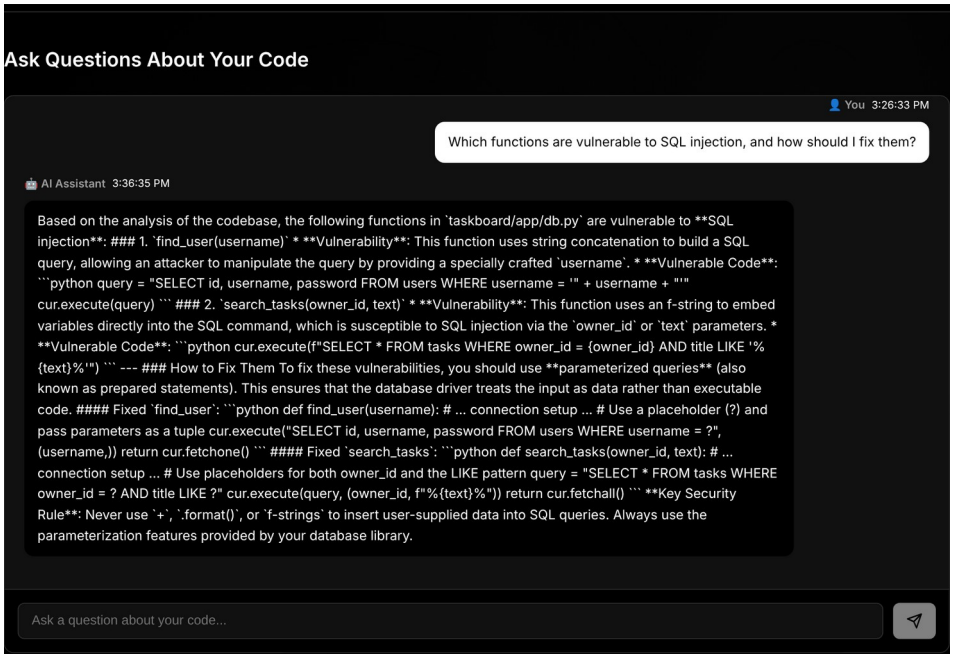


Figure 4.10: Answer to a question about SQL injection in the codebase

Figure 4.9 and Figure 4.10 show the chat panel. The question *"Which functions are vulnerable to SQL injection, and how should I fix them?"* was sent over the `/api/ws/chat/{analysis_id}` WebSocket. While the orchestrator worked, the panel showed the files it chose to re-examine and the memory updates it made. The final answer correctly identified `find_user` and `search_tasks` in `app/db.py`, quoted the vulnerable

lines, and gave corrected versions using parameterised queries. **Input:** a natural-language question. **Output:** a grounded answer with code references.

4.6.5 Command-Line Analysis

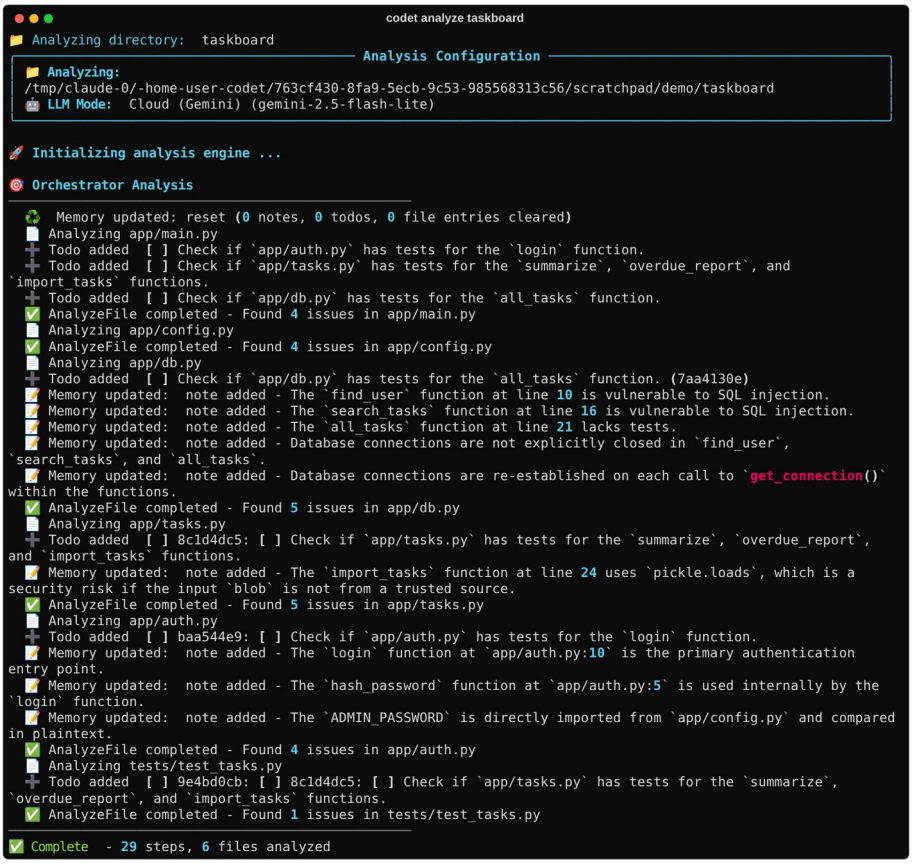


Figure 4.11: Live orchestrator events during a CLI analysis

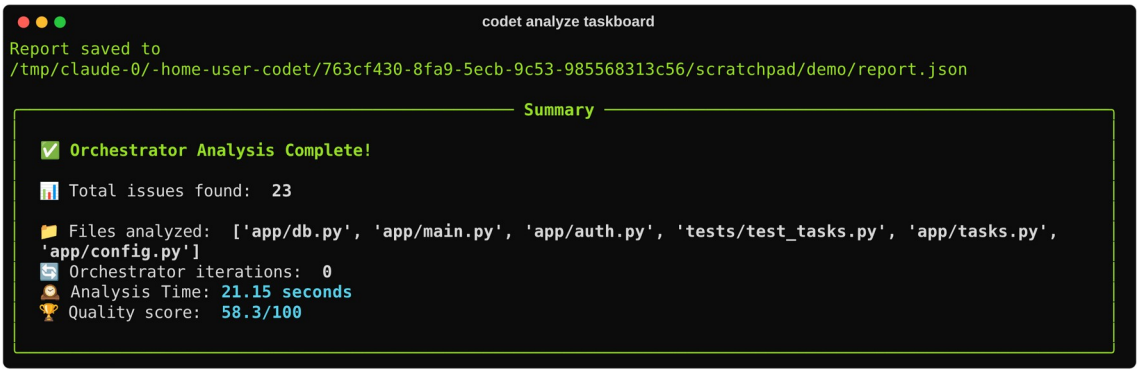


Figure 4.12: Summary printed by the CLI after the analysis

Figure 4.11 shows `codet analyze` running on the same repository from the terminal. The configuration panel shows the model in use, followed by a live log of the orchestrator's actions: each file it chose to analyse, the number of issues found, and the todos and notes written to shared memory - for example, notes that `find_user` and `search_tasks` are vulnerable to SQL injection and todos to check whether `login` and `all_tasks` have tests. Figure 4.12

shows the final summary: 23 issues in 6 files, a quality score of 58.3/100, and an analysis time of about 21 seconds. The full report was exported with `--format json --output report.json`.

4.7 Demonstration and Testing

To validate the system end to end, a small test repository named *taskboard* was written: a task-tracking service of six Python files (`app/main.py`, `app/auth.py`, `app/db.py`, `app/tasks.py`, `app/config.py` and `tests/test_tasks.py`) seeded with typical defects - SQL queries built by string concatenation, MD5 password hashing, a hard-coded secret key and admin password, `pickle` deserialisation of untrusted input, an unnecessary nested loop, unclosed database connections and very limited tests. Codet was run on it from the CLI and from the web interface. Table 4.4 summarises the two runs.

Table 4.4: Results of the demonstration runs on the taskboard repository

Metric	CLI run	Web run
Model	gemini-2.5-flash-lite	gemma-4-31b-it
Files analysed	6	6
Total issues	23	26
Critical / High	5 / 6	6 / 6
Medium / Low / Info	4 / 7 / 1	6 / 5 / 3
Security issues	7	12
Quality score	58.3 / 100	57.69 / 100

Both runs found every seeded security defect, ranked the SQL injection, hard-coded credentials and unsafe deserialisation as critical, and pointed to the correct lines. The exact counts differ because the runs used different models (selected with the `GEMINI_MODEL` setting) and because language model output is not fully deterministic; the quality scores, however, are very close. The demonstration also exposed a compatibility problem: newer Gemini models return message content as a list of parts rather than a string, which broke response parsing. This was fixed by normalising model output to plain text in `BaseAgent`.

The runs also exercised the error paths: when the free-tier API quota was exhausted or a model was temporarily overloaded, the errors were logged and the run still finished with a partial report instead of crashing, and the web interface rejected invalid GitHub URLs on the client side. Automated unit tests for the deterministic components (file filter, size checker, parsers and quality score) are planned as future work.

CHAPTER 5
CONCLUSIONS AND FUTURE SCOPE

5.1 Conclusion

This project designed and implemented **Codet**, an AI-powered code quality intelligence tool that analyses Python, JavaScript and TypeScript codebases using large language models. The project demonstrates that an orchestrator-worker multi-agent design, combined with a shared memory of notes, todos and cached results, allows a language model to analyse a repository strategically rather than file by file in isolation. Retrieval-augmented generation over a Qdrant vector database, with language-aware chunking and dual embeddings, extends the approach to repositories that are far larger than a model's context window, and the same mechanism makes it possible to apply organisation-specific coding rules written in plain Markdown.

The system produces structured, prioritised reports with categories, severities, line numbers, code snippets, suggestions and a weighted quality score, and it supports interactive, codebase-grounded question answering. It is usable both from the terminal and from a web dashboard with live progress streaming, and it can run against a cloud model or entirely on a local model for privacy. All the objectives stated in Chapter 1 were achieved.

5.2 Use of the Learned Technology in the Final Year Major Project

The technologies learned in this mini project form a strong foundation for the final-year major project. The agent framework, RAG pipeline and vector database skills are directly applicable to any system that must reason over a large private knowledge base - for example a domain-specific assistant for documents, research papers or enterprise data. The experience of building a FastAPI backend with WebSocket streaming and a React frontend can be reused for the user-facing part of the major project. Codet itself can also be extended into the major project by adding automatic fix generation, pull-request integration and quantitative evaluation against benchmark datasets.

5.3 Industrial Relevance

- **Code review automation** - software companies spend a significant share of engineering time on code review. A tool that pre-screens changes and highlights likely problems lets reviewers focus on design and business logic.
- **Security** - early detection of injection flaws, hard-coded secrets and missing validation reduces the cost of fixing vulnerabilities, which is far higher after deployment.
- **Onboarding and knowledge transfer** - the chat feature helps new team members understand unfamiliar codebases quickly.

5.4 Societal Relevance and Impact

Software now underpins banking, healthcare, education and government services, and its defects can cause financial loss and privacy breaches. By making high-quality code review more accessible, tools such as Codet can contribute to safer and more reliable software.

The project also has an educational impact: students, open-source contributors and small teams often lack experienced reviewers, and Codet's explanations act as a mentor that explains *why* code is problematic and *how* to improve it. Because it can run on free cloud tiers or local open-weight models, it is affordable for individuals and institutions, while remaining an aid to - not a replacement for - human judgement.

5.5 Limitations

- Findings are generated by a language model and may occasionally include false positives or miss issues; they must be reviewed by a developer.
- Only Python, JavaScript and TypeScript are currently supported by the parsers.

5.6 Future Scope

1. **More languages** - adding Tree-sitter grammars for Java, Go, C/C++, Rust and others.
2. **Automatic fixes** - generating patches for selected issues and opening pull requests.
3. **CI/CD and pull-request integration** - a GitHub App or Action that analyses only the changed files of each pull request and posts review comments.

REFERENCES

[1] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser and I. Polosukhin, "Attention Is All You Need," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.

[2] P. Lewis et al., "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.

[3] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks," in *Proceedings of EMNLP-IJCNLP*, 2019.

[4] Y. A. Malkov and D. A. Yashunin, "Efficient and Robust Approximate Nearest Neighbor Search Using Hierarchical Navigable Small World Graphs," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 42, no. 4, pp. 824-836, 2020.

[5] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan and Y. Cao, "ReAct: Synergizing Reasoning and Acting in Language Models," in *International Conference on Learning Representations (ICLR)*, 2023.

[6] M. Chen et al., "Evaluating Large Language Models Trained on Code," arXiv:2107.03374, 2021.

[7] Z. Feng et al., "CodeBERT: A Pre-Trained Model for Programming and Natural Languages," in *Findings of EMNLP*, 2020.

[8] A. Hindle, E. T. Barr, Z. Su, M. Gabel and P. Devanbu, "On the Naturalness of Software," in *Proceedings of the 34th International Conference on Software Engineering (ICSE)*, 2012.

[9] S. C. Johnson, "Lint, a C Program Checker," Bell Laboratories, Computing Science Technical Report 65, 1978.

[10] LangChain Documentation. [Online]. Available: https://python.langchain.com/

[11] Google AI for Developers, Gemini API Documentation. [Online]. Available: https://ai.google.dev/

[12] Qdrant Vector Database Documentation. [Online]. Available: https://qdrant.tech/documentation/

[13] FastAPI Documentation. [Online]. Available: https://fastapi.tiangolo.com/

[14] Codet source code repository. [Online]. Available: https://github.com/into-the-night/codet